Langage SQL

Stéphanie CHOLLET

Langage SQL

- Langage normalisé :
  - 1986/1987 : Première version de SQL
  - Aujourd'hui : SQL:2023
- Langage déclaratif
  - Par opposition aux langages procéduraux (C, Pascal…)

- Le langage SQL permet :
  - La définition des données
    - Gestion de la base de données
  - La manipulation des données
    - Gestion du contenu de la base de données
  - Le contrôle des données
    - Gestion des privilèges des utilisateurs
  - Le contrôle des transactions
    - Gestion de l'atomicité des requêtes



stephanie.chollet@grenoble-inp.fr

Les commandes

- Création, mise à jour, suppression de tables :
  - CREATE TABLE
  - ALTER TABLE
  - DROP TABLE
- Insertions, modifications, suppressions de lignes :
  - INSERT
  - UPDATE
  - DELETE
- Interrogation et consultation des données :
  - SELECT

stephanie.chollet@grenoble-inp.fr

Manipulation des tables

Création d'une table

```
CREATE TABLE nomTable (
        liste des noms des colonnes et du type,
        [liste des contraintes]
);
```

- Les types Oracle (pas de compatibilité stricte avec les autres bases) :
  - Chaînes de caractères : de taille fixe CHAR(taille), de taille variable VARCHAR2(taille)
  - Nombres (entiers ou réels) : NUMBER
  - Dates : DATE
- Les contraintes :
  - Clés primaires (PRIMARY KEY), clés étrangères (FOREIGN KEY … REFERENCES)
  - Unicité (UNIQUE), nullité (NULL ou NOT NULL), valeurs (CHECK), valeurs par défaut (DEFAULT)

stephanie.chollet@grenoble-inp.fr

Types de données pour PostgreSQL

- Chaînes de caractères :

CHAR(n)	The CHAR(n) datatype stores fixed-length character strings. The length n must be specified. If the stored string is shorter than the declared length, PostgreSQL pads it with spaces.
VARCHAR(n)	The VARCHAR(n) datatype stores variable-length character strings with a maximum length of n characters. If no length is specified, VARCHAR can store strings of any supported length.
TEXT	The TEXT datatype stores variable-length character strings without an explicitly declared length limit. It is suitable for storing long text.
NCHAR / NVARCHAR2	PostgreSQL does not provide separate NCHAR or NVARCHAR datatypes. Unicode characters can be stored using CHAR, VARCHAR, or TEXT, depending on the database encoding.
CLOB / NCLOB	PostgreSQL does not provide the Oracle CLOB and NCLOB datatypes. The TEXT datatype is generally used for large amounts of character data.
LONG	PostgreSQL does not provide the Oracle LONG datatype. Use TEXT for variable-length character data without an explicitly declared length limit.

stephanie.chollet@grenoble-inp.fr

Types de données pour PostgreSQL

- Numériques :

SMALLINT	The SMALLINT datatype stores small-range integers using 2 bytes. It can store values from -32,768 to 32,767.
INTEGER INT	The INTEGER (INT) datatype stores whole numbers using 4 bytes. It can store values from -2,147,483,648 to 2,147,483,647.
BIGINT	The BIGINT datatype stores large-range integers using 8 bytes. It can store values from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.
DECIMAL(p, s) NUMERIC(p, s)	The DECIMAL and NUMERIC datatypes store exact numeric values, particularly useful for financial calculations. The precision p specifies the total number of significant digits, while the scale s specifies the number of digits after the decimal point.
REAL	The REAL datatype stores approximate floating-point numbers using 4 bytes, with about 6 decimal digits of precision.
DOUBLE PRECISION	The DOUBLE PRECISION datatype stores approximate floating-point numbers using 8 bytes, with about 15 decimal digits of precision.
SERIAL BIGSERIAL	The SERIAL and BIGSERIAL types are PostgreSQL-specific conveniences for creating integer columns with automatically generated values, commonly used for identifiers. They are implemented using integer columns and sequences.

stephanie.chollet@grenoble-inp.fr

Types de données pour PostgreSQL

- Dates :

DATE	Date sans heure	`'2026-10-14'`
TIME	Heure sans date	`'14:30:00'`
TIME WITH TIME ZONE (TIMETZ)	Heure avec fuseau horaire	`'14:30:00+02'`
TIMESTAMP	Date et heure sans fuseau horaire	`'2026-10-14 14:30:00'`
TIMESTAMPTZ	Date et heure avec gestion du fuseau horaire	`'2026-10-14 14:30:00+02'`
INTERVAL	Duré ou intervalle de temps	'3 days', '2 hours'

- Booléen :

BOOLEAN	Valeur logique	`TRUE FALSE`

- …

stephanie.chollet@grenoble-inp.fr

Les contraintes

- Clés :
  - Primaire : PRIMARY KEY
  - Étrangère : FOREIGN KEY … REFERENCES
- Unicité :
  - UNIQUE
- Nullité :
  - NULL (ou NOT NULL)
- Valeurs :
  - CHECK
  - DEFAULT

stephanie.chollet@grenoble-inp.fr

Création d'une table – Exemple

$$Client(\underline{numClient}, nom, prénom, age)$$

```
CREATE TABLE Client (
        numClient CHAR(4),
        nom VARCHAR(50),
        prenom VARCHAR(50),
        age INTEGER,
        CONSTRAINT pk_Client PRIMARY KEY(numClient)
) ;
```

```
CREATE TABLE Client (
        numClient CHAR(4),
        nom VARCHAR(50) NOT NULL,
        prenom VARCHAR(50) NOT NULL,
        age INTEGER NOT NULL,
        CONSTRAINT pk_Client PRIMARY KEY(numClient),
        CONSTRAINT ck_Client_age CHECK(age > 0)
) ;
```

Une clé primaire est forcément unique et non nulle

stephanie.chollet@grenoble-inp.fr

Création de table avec des clés étrangères

- Le type des clés étrangères doit être identique au type des clés primaires référencées
  - Ex :

$$A(\underline{a1}, a2)$$

$$B(\underline{b1}, b2, \underline{a1})$$

$$B[a1] \subseteq A[a1]$$

Le type de a1 dans A et B doit être le même

- L'ordre de création des tables est important
  1. Créer les tables sans références
  2. Créer ensuite les tables avec références
     - Ex : On crée d'abord la table A et ensuite B

stephanie.chollet@grenoble-inp.fr

Création de table avec des clés étrangères - Exemple

$$Client(\underline{numClient}, nom, prénom, age)$$

$$Adhésion(\underline{numCarte}, typeAdhésion, \underline{numClientA})$$

$$Adhésion[numClientA] \subseteq Client[numClient]$$

```
CREATE TABLE Client (
        numClient CHAR(4),
        nom VARCHAR(50) NOT NULL,
        prenom VARCHAR(50) NOT NULL,
        age INTEGER NOT NULL,
        CONSTRAINT pk_Client PRIMARY KEY(numClient),
        CONSTRAINT ck_Client_dateNaiss CHECK(age > 0)
) ;
CREATE TABLE Adhesion (
        numCarte CHAR(4),
        typeAdhesion VARCHAR(50),
        numClientA CHAR(4),
        CONSTRAINT pk_Adhesion PRIMARY KEY(numCarte),
        CONSTRAINT fk_Adhesion_numClientA FOREIGN KEY(numClientA) REFERENCES Client(numClient)
) ;
```


stephanie.chollet@grenoble-inp.fr

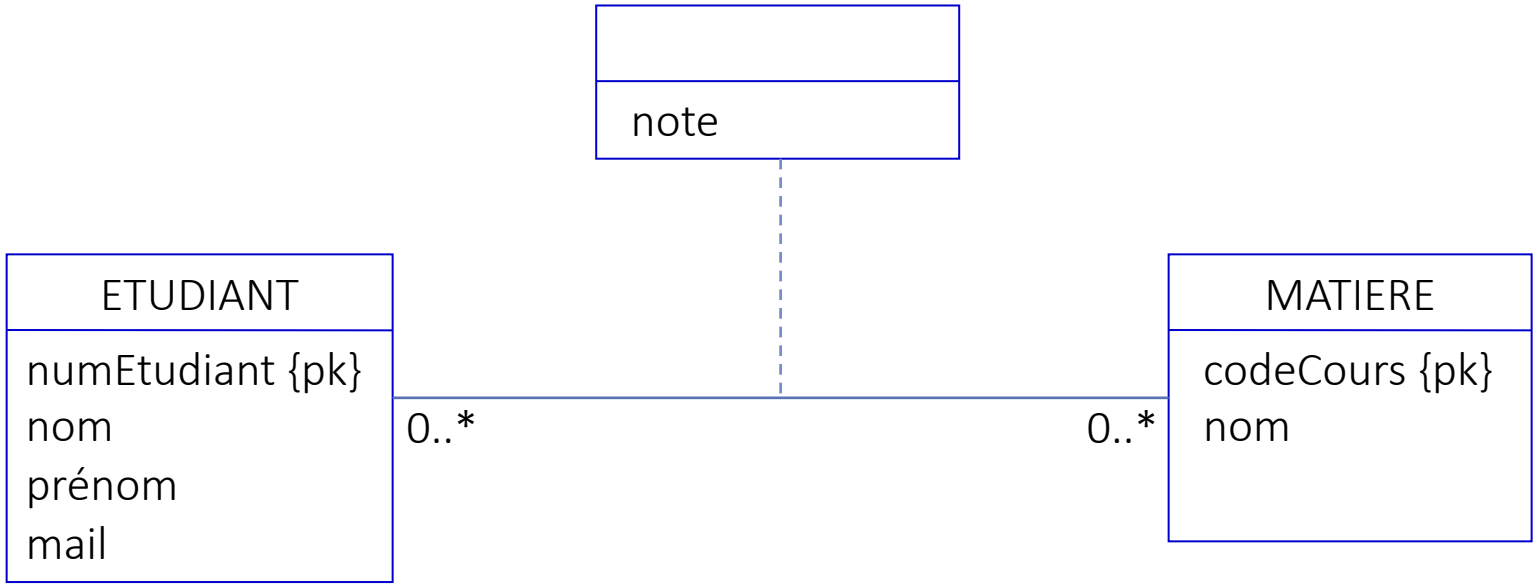
Autres contraintes

- Unicité : UNIQUE
  - attribut TYPE UNIQUE
  - CONSTRAINT u_att UNIQUE(attribut)
- Nullité : NULL (ou NOT NULL)
  - attribut TYPE NOT NULL
- Valeur :
  - attribut TYPE CHECK (attribut >= valeur)
  - CONSTRAINT ck_att CHECK (attribut >= valeur)
  - attribut TYPE DEFAULT valeur

stephanie.chollet@grenoble-inp.fr

Exemple : Evaluation des enseignements

- Modèle Entités/Associations :



- Questions :
  - Donnez le schéma relationnel
  - Donnez le code SQL

stephanie.chollet@grenoble-inp.fr

Modification d'une table – 1/2

```
ALTER TABLE nomTable ;
```

- Ajouter une colonne :

```
ALTER TABLE Produit ADD nomFournisseur VARCHAR(100);
```

- Supprimer une colonne :

```
ALTER TABLE Produit DROP nomFournisseur;
```

- Renommer une colonne :

```
ALTER TABLE Produit CHANGE prix prixTTC REAL;
```

- Changer le type de données d'une colonne :

```
ALTER TABLE Produit MODIFY designation VARCHAR(200);
```

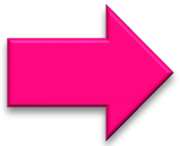

stephanie.chollet@grenoble-inp.fr

Modification d'une table – 2/2

- Ajouter une contrainte :

```
ALTER TABLE Produit ALTER prixTTC SET DEFAULT 0.0;


ALTER TABLE Produit
ADD CONSTRAINT pk_Produit PRIMARY KEY(numProduit);
```



Lire la documentation du SGBD

stephanie.chollet@grenoble-inp.fr

Suppression d'une table

```
DROP TABLE nomTable ;
```

- ATTENTION
  - Si une table est en relation avec d'autres tables, il faut d'abord supprimer les contraintes existantes dans les autres tables avant de la supprimer elle-même.



stephanie.chollet@grenoble-inp.fr

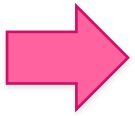
Manipulation des données

Insertion de données

```
INSERT INTO nomTable [(liste des colonnes)]
VALUES (liste des valeurs à insérer) ;
```

- Exemple : ajout d'une ligne dans la table Produit

```
INSERT INTO Produit(numProduit, designation, prix, stock)
VALUES ('1111', 'produit 1', 12.43, 15);
```



```
+------------+-------------+-------+-------+
| numProduit | designation | prix  | stock |
+------------+-------------+-------+-------+
| 1111       | produit 1   | 12.43 |    15 |
+------------+-------------+-------+-------+
```


stephanie.chollet@grenoble-inp.fr

Mise à jour de données

```
UPDATE nomTable (liste des colonnes = nouvelles valeur)
[ WHERE condition(s) sur la ligne ] ;
```

- Avant :

```
+------------+-------------+-------+-------+
| numProduit | designation | prix  | stock |
+------------+-------------+-------+-------+
| 1111       | produit 1   | 12.43 |    15 |
| 2222       | produit 2   | 45.38 |     2 |
| 3333       | produit 3   |  2.40 |    75 |
+------------+-------------+-------+-------+
```

- Requête : *Augmentation de 2 euros des prix des produits*

```
UPDATE Produit SET prix = prix + 2 ;
```

- Après :

```
+------------+-------------+-------+-------+
| numProduit | designation | prix  | stock |
+------------+-------------+-------+-------+
| 1111       | produit 1   | 14.43 |    15 |
| 2222       | produit 2   | 47.38 |     2 |
| 3333       | produit 3   |  4.40 |    75 |
+------------+-------------+-------+-------+
```


stephanie.chollet@grenoble-inp.fr

Suppression de données

```
DELETE FROM nomTable
[ WHERE condition(s) sur la ligne ] ;
```

- Avant :

```
+------------+-------------+-------+-------+
| numProduit | designation | prix  | stock |
+------------+-------------+-------+-------+
| 1111       | produit 1   | 14.43 |    15 |
| 2222       | produit 2   | 47.38 |     2 |
| 3333       | produit 3   |  4.40 |    25 |
+------------+-------------+-------+-------+
```

- Requête : *Vider la table Produit*

```
DELETE FROM Produit ;
```

- Après : La table est vide

stephanie.chollet@grenoble-inp.fr

Interrogation des données

Projection, sélection, jointure,
opérations ensemblistes et division

Opération SELECT : principe de base

- Structure de base :

```
SELECT <liste des attributs>
FROM <liste des tables>
WHERE <condition(s) sur un ou plusieurs attributs>
```

- Equivalence avec l'algèbre relationnelle pour une requête simple :

$$\pi_{a_1,a_2,\ldots,a_n}\left(\sigma_P(R_1 \times R_2 \times \cdots \times R_m)\right) \Leftrightarrow$$

```
SELECT DISTINCT a1, a2, …, an
FROM R1, R2, …, Rm
WHERE P;
```


stephanie.chollet@grenoble-inp.fr

Projection (π)

- Soit Employé(mat, nom, sal)
  - Requête : informations sur tous les employés

$$\pi_{nom,\, mat,\, sal}(Employe)$$

```
SELECT DISTINCT nom, mat, sal
FROM Employe;
```

⟺

```
SELECT DISTINCT *
FROM Employe;
```

  - Requête : noms de tous les employés

$$\pi_{nom}(Employe)$$

```
SELECT DISTINCT nom
FROM Employe;
```

Contrairement à l'algèbre relationnelle, SQL n'élimine pas les doublons, il faut utiliser le mot **DISTINCT**.

→ Les colonnes apparaissent dans l'ordre précisé dans la clause SELECT

→ Les lignes apparaissent dans l'ordre de stockage physique

stephanie.chollet@grenoble-inp.fr

Sélection (σ)

- Conditions de sélection :
  - =, !=, <>, > , >=, <, <=
  - BETWEEN, LIKE
  - IS NULL, IS NOT NULL, IN

- Exemple :
  - Soit Employé(mat, nom, job, sal, sexe)
  - Requête : Donner le nom, le job, le salaire et le sexe des employés ayant un salaire supérieur à 1400€

$$\pi_{nom,\, job,\, sal,\, sexe}(\sigma_{sal>1400}(Employe)) \Longleftrightarrow$$

```
SELECT nom, job, sal, sexe
FROM Employe
WHERE sal > 1400;
```


stephanie.chollet@grenoble-inp.fr

Conditions de sélection – 1/2

- BETWEEN : spécifie une plage de valeurs

```
SELECT nom FROM Employe
WHERE sal BETWEEN 1200 AND 1600;
```



```
SELECT nom FROM Employe
WHERE sal >= 1200
AND sal <= 1600;
```

- LIKE : recherche d'une chaîne de caractères à l'intérieur des valeurs d'une expression comprenant éventuellement des caractères jokers (_, %)

```
SELECT job FROM Employe
WHERE nom LIKE '_A%';
```

- IN : spécifie une liste de valeurs

```
SELECT nom FROM Employe
WHERE sal IN(1200, 1600);

SELECT * FROM Employe
WHERE (nom, prenom) IN(('TOTO', 'titi'), ('TATA', 'titi'));
```


stephanie.chollet@grenoble-inp.fr

Conditions de sélection – 2/2



- L'absence de valeur dans une colonne d'une ligne d'une table est assimilée à une valeur nulle

- IS NULL : permet d'extraire les lignes dont certaines colonnes n'ont pas été renseignées

```
SELECT nom FROM Employe
WHERE sal IS NULL ;
```

- IS NOT NULL : variante pour la négation

```
SELECT nom FROM Employe
WHERE sal IS NOT NULL ;
```


stephanie.chollet@grenoble-inp.fr

Jointures simples : (⋈) et (×)

- Produit cartésien :
  - Avec Employé(mat, nom, numdep) et Département(numdep, matchef)

```
SELECT mat, nom, e.numdep, d.numdep, matchef
FROM Employe e, Departement d ;
```

- Jointure naturelle (ou équi-jointure)

```
SELECT mat, nom, e.numdep, d.numdep, matchef
FROM Employe e, Departement d
WHERE e.numdep = d.numdep ;

SELECT nom, sal, nomdep
FROM Employe, Departement
WHERE mat = matchef ;
```


stephanie.chollet@grenoble-inp.fr

Jointures en SQL2

Opération	Algèbre relationnelle	SQL2
Produit cartésien	$R_1 \times R_2$	R1 CROSS JOIN R2
Thêta-jointure (ou θ-jointure)	$R_1 \bowtie_{R_1.A<R_2.B} R_2$	R1 JOIN R2 ON prédicat (R1 JOIN R2 ON R1.A < R2.B)
Thêta-jointure externe	$R_1 \ltimes_{R_1.A<R_2.B} R_2$ $R_1 \rtimes_{R_1.A<R_2.B} R_2$	R1 FULL/LEFT/RIGHT OUTER JOIN R2 ON prédicat
Jointure naturelle	$R_1 \bowtie R_2$	R1 NATURAL JOIN R2
Jointure externe	$R_1 \ltimes R_2$ $R_1 \rtimes R_2$	R1 NATURAL FULL/LEFT/RIGHT OUTER JOIN R2

stephanie.chollet@grenoble-inp.fr

Jointure naturelle

Employé

empno	depno	sal
Tom	1	10000
Jim	2	20000
Karim	3	15000

Département

depno	dname	loc
1	Commercial	Paris
2	Administration	Valence
4	Technique	Valence

- $Employé \bowtie Département$:

empno	depno	sal	dname	loc
Tom	1	10000	Commercial	Paris
Jim	2	20000	Administration	Valence

```
SELECT empno, e.depno, sal, dname, loc
FROM Employe e, Departement d
WHERE e.numdep = d.numdep ;

SELECT empno, depno, sal, dname, loc
FROM Employe e NATURAL JOIN Departement d
```


stephanie.chollet@grenoble-inp.fr

Jointure externe

Employé

empno	depno	sal
Tom	1	10000
Jim	2	20000
Karim	3	15000

Département

depno	dname	loc
1	Commercial	Paris
2	Administration	Valence
4	Technique	Valence

- $Employé \ltimes Département$

empno	depno	sal	dname	loc
Tom	1	10000	Commercial	Paris
Jim	2	20000	Administration	Valence
Karim	3	15000	NULL	NULL

```
SELECT empno, e.depno, sal, dname, loc
FROM Employe e NATURAL LEFT OUTER JOIN Departement d
```


stephanie.chollet@grenoble-inp.fr

Opérations ensemblistes (∪), (∩) et (−)

- UNION, INTERSECT, EXCEPT

```
SELECT A, B
FROM R
UNION
SELECT A, B
FROM S ;
```

```
SELECT A, B
FROM R
INTERSECT
SELECT A, B
FROM S ;
```

```
SELECT A, B
FROM R
EXCEPT
SELECT A, B
FROM S ;
```


stephanie.chollet@grenoble-inp.fr

Division (÷)

- Réécriture de la division :
  - Avec *Z* l'ensemble des attributs de *R*
  - Avec *X* l'ensemble des attributs de *S* tels que $X \subseteq Y$
  - On a $Y = Z - X$

$$\pi_Y(R) - \pi_Y((\pi_Y(R) \times S) - R)$$

- Exemple :
  - Avec schema(r_1) = {etud, dep} et schema(r_2) = {dep}

$$r_1 \div r_2 = \pi_{etud}(r_1) - \pi_{etud}((\pi_{etud}(r_1) \times r_2) - r_1)$$

stephanie.chollet@grenoble-inp.fr

Division (÷)

$$r_1 \div r_2 = \pi_{etud}(r_1) - \pi_{etud}((\pi_{etud}(r_1) \times r_2) - r_1)$$

r_1	etud	dep
	1	1
	1	2
	1	3
	1	4
	2	1
	2	2
	3	2
	4	2
	4	4

r_2	dep
	2

1

$\pi_{etud}(r_1)$	etud
	1
	2
	3
	4

2

$\pi_{etud}(r_1) \times r_2$	etud	dep
	1	2
	2	2
	3	2
	4	2

3

$(\pi_{etud}(r_1) \times r_2) - r_1$	etud	dep
	~~1~~	~~2~~
	~~2~~	~~2~~
	~~3~~	~~2~~
	~~4~~	~~2~~

4

$\pi_{etud}((\pi_{etud}(r_1) \times r_2) - r_1)$	etud

5

$r_1 \div r_2$	etud
	1
	2
	3
	4

stephanie.chollet@grenoble-inp.fr

Division (÷)

$$r_1 \div r_2 = \pi_{etud}(r_1) - \pi_{etud}((\pi_{etud}(r_1) \times r_2) - r_1)$$

- En SQL :

```
SELECT DISTINCT r1.etud
FROM r1
EXCEPT
SELECT DISTINCT etud
FROM ((SELECT DISTINCT r1.etud, r2.dep
       FROM r1, r2)
       EXCEPT
       SELECT r1.etud, r1.dep
       FROM r1);
```


stephanie.chollet@grenoble-inp.fr

Imbrication de requêtes

ANY, ALL, EXISTS

Imbrications simples – 1/2

- Equivalence pour la jointure naturelle :

```
SELECT e.nom, e.sal
FROM Employe e, Departement d
WHERE e.numdep = d.numdep
AND d.budget > 500000) ;
```

```
SELECT nom, sal
FROM Employe
WHERE numdep IN (SELECT numdep
                 FROM Departement
                 WHERE budget > 500000) ;
```


stephanie.chollet@grenoble-inp.fr

Imbrications simples – 2/2

- Equivalence pour la différence :

```
SELECT depno
FROM Departement
MINUS
SELECT depno
FROM Employe ;
```

```
SELECT depno
FROM Departement
WHERE depno NOT IN (SELECT DISTINCT depno
                    FROM Employe) ;
```


stephanie.chollet@grenoble-inp.fr

Imbrications complexes – 1/3

- ANY : quantifieur existentiel

```
SELECT ...
FROM R
WHERE X op ANY (SELECT Y
                FROM S
                WHERE ...) ;
```

  - Avec op un opérateur binaire de comparaison classique :
    =, >, < , <>, … qui retourne vrai ou faux

  - Signification : Afin qu'une ligne satisfasse au critère défini dans la requête externe, la valeur spécifiée dans la colonne qui introduit la sous-requête doit être *(op)* à au moins une valeur de la liste retournée par la sous-requête.

stephanie.chollet@grenoble-inp.fr

Imbrications complexes – 2/3

- ALL :

```
SELECT ...
FROM R
WHERE X op ALL (SELECT Y
                FROM S
                WHERE ...) ;
```

  - Avec op un opérateur binaire de comparaison classique :
    =, >, < , <>, … qui retourne vrai ou faux

  - Signification : Afin qu'une ligne satisfasse au critère défini dans la requête externe, la valeur spécifiée dans la colonne qui introduit la sous-requête doit être *(op)* à toutes les valeurs de la liste retournée par la sous-requête.

stephanie.chollet@grenoble-inp.fr

Imbrications complexes – 3/3



- EXISTS (NOT EXISTS) :

```
SELECT X
FROM R
WHERE EXISTS (SELECT *
              FROM S
              WHERE C) ;
```

- Signification : La clause WHERE de la requête externe teste l'existence des lignes retournées par la sous-requête. La sous-requête ne génère pas de données, elle retourne TRUE ou FALSE.

stephanie.chollet@grenoble-inp.fr

Fonctions de calculs et opérateurs d’agrégation

MAX, MIN, AVG, SUM, COUNT

GROUP BY, HAVING, ORDER BY

Fonctions de calculs



- COUNT : nombre de lignes non nulles
- MIN, MAX, SUM, AVG, STD, VARIANCE : opérations sur les valeurs

```
SELECT SUM(stock * prixTTC)
FROM produit;
```

```
+-----------------------+
| SUM(stock * prixTTC) |
+-----------------------+
|               1230.29 |
+-----------------------+
```


stephanie.chollet@grenoble-inp.fr

Opérations d'agrégation

- GROUP BY : précise les attributs de partitionnement des tables déclarées dans la clause FROM

- Exemple : Nombre de fournisseurs par ville

Ville	Fnom
Valence	Durand
Valence	Dupont
Paris	Bernard
Paris	Martin
Paris	Bernard

```
SELECT Ville, COUNT(Fnom)
FROM Fournisseur
GROUP BY Ville ;
```

Ville	COUNT(Fnom)
Valence	2
Paris	3

L'attribut ou les attributs qui apparaissent dans le SELECT hors fonction d'agrégation doivent apparaître dans le GROUP BY

stephanie.chollet@grenoble-inp.fr

Opérations d'agrégation

- HAVING : permet d'éliminer des lignes d'un partitionnement
  - (équivalent du WHERE mais pour un partitionnement !)

- Exemple : Nombre de fournisseurs par ville qui ont au moins 3 fournisseurs

Ville	Fnom
Valence	Durand
Valence	Dupont
Paris	Bernard
Paris	Martin
Paris	Bernard

```
SELECT Ville, COUNT(Fnom)
FROM Fournisseur
GROUP BY Ville
HAVING COUNT(Fnom) > 2;
```

Ville	COUNT(Fnom)
~~Valence~~	~~2~~
Paris	3

stephanie.chollet@grenoble-inp.fr

Opérations d'agrégation

- Imbrication d'opérations d'agrégation
  - SELECT AGG1(AGG2 [DISTINCT] attribut))
  - Où AGG1 est MIN, MAX ou AVG qui s'applique au résultat de AGG2 sur l'attribut
  - Et AGG2 est associé à un GROUP BY

- Exemple : Nombre moyen de fournisseurs par ville

Ville	Fnom
Valence	Durand
Valence	Dupont
Paris	Bernard
Paris	Martin
Paris	Bernard

```
SELECT AVG(COUNT(Fnom))
FROM Fournisseur
GROUP BY Ville;
```

AVG(COUNT(Fnom))
2,5

stephanie.chollet@grenoble-inp.fr

Opérations d'agrégation

- Exemple : Nom des villes avec leur nombre de fournisseur dont le nombre de fournisseurs est inférieur au nombre moyen de fournisseur par ville.

```
SELECT Ville, COUNT(Fnom)
FROM Fournisseur
GROUP BY Ville ;
```

Ville	COUNT(Fnom)
Valence	2
Paris	3

```
SELECT AVG(COUNT(Fnom))
FROM Fournisseur
GROUP BY Ville;
```

AVG(COUNT(Fnom))
2,5

stephanie.chollet@grenoble-inp.fr

Opérations d'agrégation

- Exemple : Nom des villes avec leur nombre de fournisseur dont le nombre de fournisseurs est inférieur au nombre moyen de fournisseur par ville.

```
SELECT Ville, COUNT(Fnom)
FROM Fournisseur
GROUP BY Ville
HAVING COUNT(Fnom) <= (SELECT AVG(COUNT(Fnom))
                       FROM Fournisseur
                       GROUP BY Ville);
```

Ville	COUNT(Fnom)
Valence	2



Impossible tel quel avec PostgreSQL !

stephanie.chollet@grenoble-inp.fr

Opération de tri

- ORDER BY : tri les résultats par ordre ascendant (ASC) ou descendant (DESC)

- Exemple :

```
SELECT *
FROM produit
ORDER BY prix DESC ;
```

```
+------------+-------------+-------+-------+
| numProduit | designation | stock | prix  |
+------------+-------------+-------+-------+
| 3333       | produit3    |     4 | 62.41 |
| 4444       | produit4    |    20 | 34.02 |
| 1111       | produit1    |    15 | 12.43 |
| 2222       | produit2    |    20 |  5.69 |
+------------+-------------+-------+-------+
```


stephanie.chollet@grenoble-inp.fr

Synthèse de la commande SELECT

```
SELECT <liste des attributs>
FROM <liste des tables>
[WHERE <condition(s) sur un ou plusieurs attributs>]
[GROUP BY <liste des attributs de regroupement>]
[HAVING <conditions pour les regroupements>]
[ORDER BY <liste des attributs ordonnés [ASC | DESC]>];
```


stephanie.chollet@grenoble-inp.fr